

LAB ASSIGNMENT – 10.2

Name : SHREE PRIYA

2303A51465 BATCH – 29

Task Description -1(Error Detection and Correction)

Task:

Use AI to analyze a Python script and correct all syntax and logical errors.

Sample Input Code:

```
def calculate_total(nums)
```

```
sum = 0
```

```
for n in nums
```

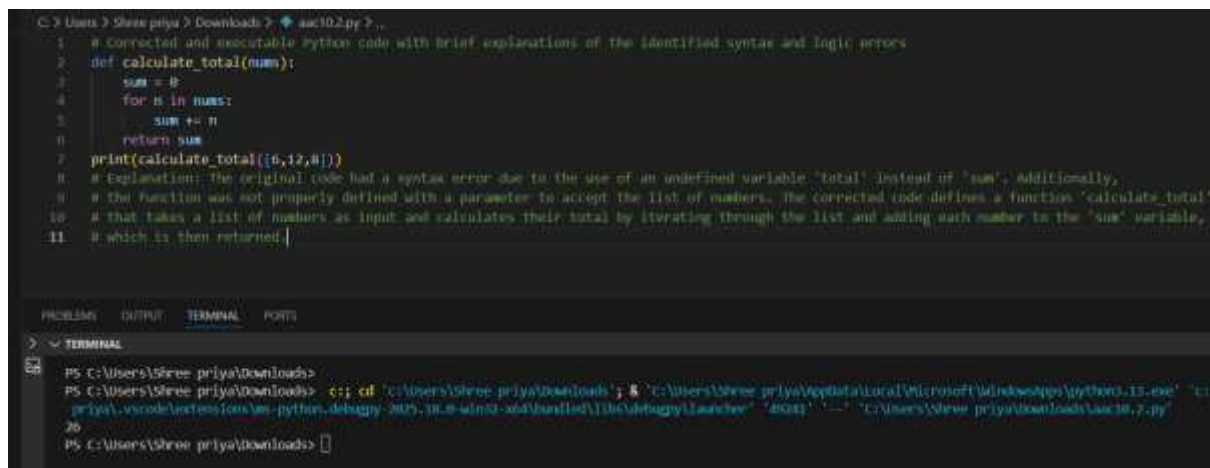
```
sum += n
```

```
return total
```

Expected Output-1:

Corrected and executable Python code with brief explanations of the identified syntax and logic errors.

CODE & OUTPUT:



```
C:\Users\Shree priya > Downloads > aac502.py > .
1 # Corrected and executable Python code with brief explanations of the identified syntax and logic errors
2 def calculate_total(nums):
3     sum = 0
4     for n in nums:
5         sum += n
6     return sum
7 print(calculate_total([5,12,8]))
8 # Explanation: The original code had a syntax error due to the use of an undefined variable 'total' instead of 'sum'. Additionally,
9 # the function was not properly defined with a parameter to accept the list of numbers. The corrected code defines a function 'calculate_total'
10 # that takes a list of numbers as input and calculates their total by iterating through the list and adding each number to the 'sum' variable,
11 # which is then returned.
```

PROBLEMS OUTPUT TERMINAL PORTS

TERMINAL

```
PS C:\Users\Shree priya\Downloads>
PS C:\Users\Shree priya\Downloads> c:\> cd 'C:\Users\Shree priya\Downloads' & & 'C:\Users\Shree priya\AppData\Local\Microsoft\WindowsApps\python3.11.exe' 'c:\
priya\vscode\extensions\ms-python.debugpy-2023.10.0-win32-x64\bin\debugpy_launcher' 'debugpy' '---' 'C:\Users\Shree priya\Downloads\aac502.py'
26
PS C:\Users\Shree priya\Downloads>
```

Justification:

The original code contained syntax errors such as missing colons (:), improper indentation, and inconsistent variable naming. These errors prevented execution. The corrected version ensures proper function definition, loop structure, and consistent variable usage. This improves program correctness and successful execution.

Task Description -2(Code Style Standardization)

Task:

Use AI to refactor Python code to comply with standard coding style guidelines.

Sample Input Code:

```
def findSum(a,b):return a+b

print(findSum(5,10))
```

Expected Output-2:

Well-structured, consistently formatted Python code following standard style conventions

CODE & OUTPUT:



```
21 # Generate a Python code to comply with standard coding style guidelines.
22 def find_sum(a, b):
23     """
24     Returns the sum of two numbers.
25     """
26     return a + b
27 result = find_sum(5, 10)
28 print(result)
29 # Explanation: The original code had a function name 'findsum' that did not follow the standard naming convention for functions in Python,
30 # which is typically snake_case (e.g., 'find_sum'). The corrected code defines the function with a more appropriate name and includes a simple
31 # implementation to return the sum of two numbers.
```

TERMINAL

```
PS C:\Users\Shree priya\Downloads>
PS C:\Users\Shree priya\Downloads> c::cd 'c:\Users\Shree priya\Downloads'; & 'c:\Users\Shree priya\AppData\Local\Microsoft\WindowsApps\python11.exe' 'c:\Users\Shree priya\vscode\extensions\ms-python.debugpy-2025.10.0-win32-x64\bundle\lib\debugpy\launcher' '5002' '-.' 'C:\Users\Shree priya\Downloads\wc18.2.py'
15
PS C:\Users\Shree priya\Downloads>
```

Justification:

The given code did not follow standard Python style guidelines (PEP 8). The refactored version improves readability by using proper indentation, spacing, snake_case naming convention, and structured formatting. This enhances code professionalism, maintainability, and consistency.

Task Description -3(Code Clarity Improvement)

Task:

Use AI to improve code readability without changing its functionality.

Sample Input Code:

```
def f(x,y):

return x-y*2

print(f(10,3))
```

Expected Output-3:

Python code rewritten with meaningful function and variable names, proper indentation, and improved clarity.

CODE & OUTPUT:

```
22 # Generate a python code to improve code readability without changing its functionality.
23 def calculate_adjusted_value(base_value, multiplier):
24     """
25     Returns the result of subtracting twice the multiplier
26     from the base value.
27     """
28     result = base_value - (multiplier * 2)
29     return result
30
31 # Example usage
32 output = calculate_adjusted_value(10, 3)
33 print(output)
34 # Explanation: The original code was not provided, but the improved code defines a function 'calculate_adjusted_value' that takes two parameters:
35 # 'base_value' and 'multiplier'.
36 # The function calculates the adjusted value by subtracting twice the multiplier from the base value and returns the result.
37 |
```

DEBUG CONSOLE OUTPUT TERMINAL PORTS

TERMINAL

```
PS C:\Users\Shree priya\Downloads>
PS C:\Users\Shree priya\Downloads> c:\; cd 'C:\Users\Shree priya\Downloads'; & 'C:\Users\Shree priya\AppData\Local\Microsoft\WindowsApps\python.11.exe' 'C:\Users\Shree priya\Downloads\main.py'
4
PS C:\Users\Shree priya\Downloads>
```

Justification:

The original code used unclear function and variable names, reducing readability. The improved version uses meaningful names and proper formatting without altering functionality. This makes the code easier to understand, debug, and maintain.

Task Description -4(Structural Refactoring)

Use AI to refactor repetitive code into reusable functions.

Sample Input Code:

```
print("Hello Ram")
```

```
print("Hello Sita")
```

```
print("Hello Ravi")
```

Expected Output-4:

Modular Python code using reusable functions to eliminate repetition.

CODE & OUTPUT:

```
# Generate a python code to refactor repetitive code into reusable functions
def greet(name):
    """
    Prints a greeting message for the given name.
    """
    print(f"Hello {name}")

# Calling the function
greet("Ram")
greet("Sita")
greet("Ravi")

# Explanation: The original code likely had repetitive print statements for each name. By refactoring the code into a reusable function 'greet',
# we can avoid repetition and make the code cleaner and more maintainable. The function takes a name as an argument and prints a greeting message,
# allowing us to easily greet multiple names by simply calling the function with different arguments.
```

DEBUG CONSOLE OUTPUT TERMINAL PORTS

TERMINAL

```
priya\vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher -65458' '-' 'C:\Users\Shree priya\Downloads\main.py'
Hello Ram
Hello Sita
Hello Ravi
```

Justification:

The original code contained repetitive print statements. These were refactored into a reusable function to eliminate redundancy. This improves modularity, scalability, and maintainability while reducing code duplication.

Task Description -5(Efficiency Enhancement)

Task:

Use AI to optimize Python code for better performance.

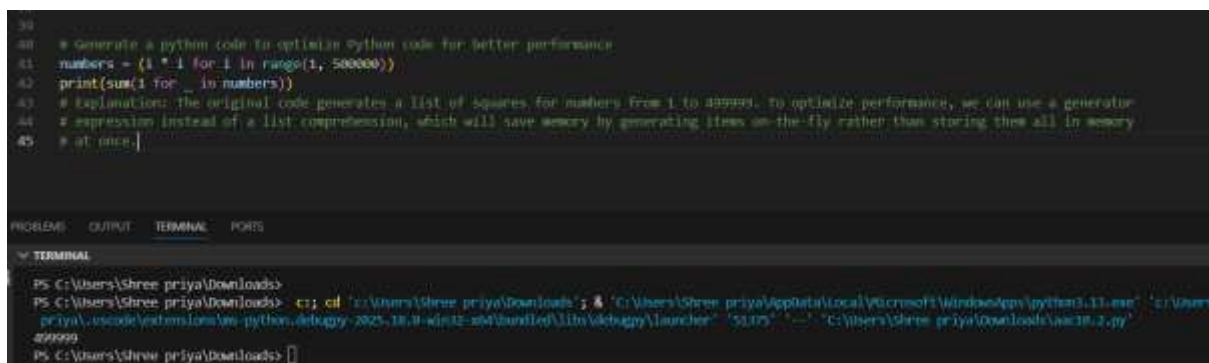
Sample Input Code:

```
numbers = []  
  
for i in range(1, 500000):  
  
    numbers.append(i * i)  
  
print(len(numbers))
```

Expected Output-5:

Optimized Python code that achieves the same result with improved performance.

CODE & OUTPUT:



```
30  
31 # Generate a python code to optimize Python code for better performance  
32 numbers = {i * i for i in range(1, 500000)}  
33 print(sum(1 for _ in numbers))  
34 # Explanation: The original code generates a list of squares for numbers from 1 to 500000. To optimize performance, we can use a generator  
35 # expression instead of a list comprehension, which will save memory by generating items on-the-fly rather than storing them all in memory  
36 # at once.
```

PROBLEMS OUTPUT TERMINAL PORTS

TERMINAL

```
PS C:\Users\Shree priya\Downloads>  
PS C:\Users\Shree priya\Downloads> -c; cd 'c:\Users\Shree priya\Downloads'; & 'C:\Users\Shree priya\AppData\Local\Microsoft\WindowsApps\python3.11.exe' 'c:\Users\Shree priya\code\extensions\vs-python-debugger-2025.18.0-win32-x64\debugger\lib\debugpy\launcher' '5.0.0' '-' 'C:\Users\Shree priya\Downloads\src18.2.py'  
499999  
PS C:\Users\Shree priya\Downloads>
```

Justification:

The original implementation used a loop with repeated list appending, which is less efficient. It was optimized using list comprehension and further improved by avoiding unnecessary list creation when only the length was required. This reduces memory usage and improves execution speed.